UNIT 5 · TOPIC 5.2

Protecting Applications and Data: Managerial Controls and Access Controls

Assignment, Answer Key

Teacher answer key, 27 points total.

Practice Questions

5.2.a · Access control models (RBAC, DAC, MAC)

1. In Unix, a file's owner sets read/write/execute permissions for owner, group, and others. Which access control model does this describe? (1 pt)

- ● **A. Discretionary Access Control (DAC)**
- ○ B. Mandatory Access Control (MAC)
- ○ C. Role-Based Access Control (RBAC)
- ○ D. Attribute-Based Access Control (ABAC)

Why: The object's owner decides who else may access it, that discretion is the defining trait of DAC.

2. Select ALL statements that correctly describe Mandatory Access Control (MAC). (3 pts)

- ■ **A. A central policy authority assigns labels to subjects and objects**
- □ B. Individual object owners can freely grant access to anyone they choose
- ■ **C. The system enforces label-based rules automatically, regardless of owner preference**
- ■ **D. MAC originated in government/military contexts (e.g., Bell-LaPadula)**

Why: MAC removes discretion from owners; enforcement is centralized and automatic based on labels.

3. A hospital assigns permissions to a 'Nurse' role and a 'Billing Clerk' role rather than to each of its 5,000 staff individually. Which model is this? (1 pt)

- ○ A. DAC
- ○ B. MAC
- ● **C. RBAC**
- ○ D. None of these

Why: Permissions attached to named roles, with users assigned to roles, is Role-Based Access Control.

4. Name the design principle: grant each subject only the permissions needed to do its job, nothing more. (1 pt)

Acceptable answers: least privilege; principle of least privilege

Why: Least privilege limits the blast radius of a compromised account.

5.2.b · Roles, permissions, and least privilege

5. What is the principle that a subject should be granted only the permissions it needs to do its job, nothing more? (1 pt)

- ● **A. Least privilege**
- ○ B. Separation of duties
- ○ C. Role recertification
- ○ D. Mandatory access control

Why: Least privilege minimizes the blast radius of a compromised or misused account.

6. Select ALL statements that correctly describe privilege creep. (3 pts)

■ **A. It is the gradual, unnoticed accumulation of unnecessary permissions over time**

■ **B. It happens because missing permissions are loudly noticed and fixed, while excess permissions are silently ignored**

□ C. It is prevented automatically by RBAC with no further effort

■ **D. Periodic role review/recertification is used to counteract it**

Why: RBAC organizes permissions but does not by itself prevent creep; deliberate review is required.

7. Why should the person who creates a vendor payment not also be the person who approves it? (1 pt)

● **A. Separation of duties requires that a sensitive process need collusion between two people to be abused**

○ B. It violates least privilege by definition

○ C. Payments cannot be created and approved by the same software

○ D. It is a MAC labeling requirement

Why: Separation of duties splits sensitive processes across roles so a single compromised or malicious account cannot complete them alone.

8. Name the metaphor used to describe how much damage a compromised account with excess permissions can do. (1 pt)

Acceptable answers: blast radius

Why: Least privilege caps the 'blast radius' of a compromised identity.

5.2.c · Credential theft prevention and lateral movement

9. An attacker who compromises one marketing laptop uses stolen credentials to hop to several other internal machines in search of valuable systems. What is this process called? (1 pt)

● **A. Lateral movement**

○ B. Cross-site scripting

○ C. Broken object level authorization

○ D. Data exfiltration

Why: Moving from an initial foothold to additional systems within the network is lateral movement.

10. Select ALL statements that correctly describe pass-the-hash / pass-the-ticket attacks. (3 pts)

■ **A. The attacker authenticates using a stolen hash or ticket without ever knowing the plaintext password**

□ B. These attacks require the attacker to crack the password first

■ **C. They exploit authentication protocols that accept hash- or ticket-based authentication**

■ **D. Multi-factor authentication significantly reduces the impact of a successfully stolen credential**

Why: Pass-the-hash/ticket attacks specifically avoid needing the plaintext password; MFA blunts the value of any stolen credential.

11. Why does reusing the same local administrator password across every machine in an organization make lateral movement much easier for an attacker? (1 pt)

○ A. It has no effect on lateral movement

○ B. Shared passwords are always encrypted differently on each machine

● **C. One stolen admin credential authenticates on every machine that shares it, removing the need to steal new credentials at each hop**

○ D. It only affects DAC systems

Why: A single shared admin credential becomes a skeleton key across the whole environment once stolen.

12. Name one behavioral signal defenders watch for that suggests an account is engaged in lateral movement (e.g., unusually high count of what). (1 pt)

Acceptable answers: distinct hosts; number of hosts; host count; many hosts

Why: A single account authenticating to an unusually large number of distinct hosts in a short window is a classic lateral-movement signal.

Coding Assignments (Python 3)

5.2.a · Access control models (RBAC, DAC, MAC)

3 pts · auto-graded on the site

Coding problem, 5.2.a: RBAC / DAC / MAC access decision

Skill: Mitigate Risk · **Maps to:** 5.2.A · **Language:** Python 3 (pytest)

Problem

Build the access-decision function a policy engine would run for a fictional file system that supports all three access control models. Given a request (subject, object, action, and the object's model), decide whether the request is ALLOWed or DENYed.

Rules by model

- `"dac"`, allowed if the subject is the object's `owner`, OR the subject appears in the object's `grants` list for that action (e.g., `grants: {"read": ["bob"], "write": []}`).
- `"mac"`, allowed only if the subject's clearance level is >= the object's classification level. Levels (low to high): `"public"` < `"confidential"` < `"secret"` < `"top_secret"`.
- `"rbac"`, allowed if any of the subject's `roles` is present in the object's `allowed_roles` list for the requested action (e.g., `allowed_roles: {"read": ["nurse","admin"], "write": ["admin"]}`).

If the object's `model` is not one of `"dac"`, `"mac"`, `"rbac"`, deny with reason `"unknown_model"`.

Function contract

Implement `solve(request: dict) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`request`: a dict with keys `"subject"` (dict) and `"object"` (dict) and `"action"` (str). See `interface.md` for the exact shape per model.

Output

`{"allowed": bool, "reason": str}` where reason is one of `"owner"`, `"granted"`, `"dac_denied"`, `"clearance_sufficient"`, `"clearance_insufficient"`, `"role_match"`, `"rbac_denied"`, `"unknown_model"`.

Examples

```
DAC owner:
  {"subject":{"name":"alice"},"object":{"model":"dac","owner":"alice","grants":{}},"action":"read"}
  -> {"allowed": True, "reason": "owner"}

MAC insufficient clearance:
```

```
  {"subject":{"clearance":"confidential"},"object":{"model":"mac","classification":"secret"},"action":"read"}
  -> {"allowed": False, "reason": "clearance_insufficient"}

RBAC role match:

{"subject":{"roles":["nurse"]},"object":{"model":"rbac","allowed_roles":{"read":["nurse","admin"]}},"action":"re
ad"}
  -> {"allowed": True, "reason": "role_match"}
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import Dict

_LEVELS = {"public": 0, "confidential": 1, "secret": 2, "top_secret": 3}


def solve(request: Dict[str, object]) -> Dict[str, object]:
    subject = request["subject"]
    obj = request["object"]
    action = request["action"]
    model = obj.get("model")

    if model == "dac":
        if subject.get("name") == obj.get("owner"):
            return {"allowed": True, "reason": "owner"}
        grants = obj.get("grants", {})
        if subject.get("name") in grants.get(action, []):
            return {"allowed": True, "reason": "granted"}
        return {"allowed": False, "reason": "dac_denied"}

    if model == "mac":
        sub_level = _LEVELS.get(subject.get("clearance"), -1)
        obj_level = _LEVELS.get(obj.get("classification"), 99)
        if sub_level >= obj_level:
            return {"allowed": True, "reason": "clearance_sufficient"}
        return {"allowed": False, "reason": "clearance_insufficient"}

    if model == "rbac":
        allowed_roles = obj.get("allowed_roles", {}).get(action, [])
        subject_roles = subject.get("roles", [])
        if any(r in allowed_roles for r in subject_roles):
            return {"allowed": True, "reason": "role_match"}
        return {"allowed": False, "reason": "rbac_denied"}

    return {"allowed": False, "reason": "unknown_model"}
```

5.2.b · Roles, permissions, and least privilege

3 pts · auto-graded on the site

Coding problem, 5.2.b: Detect least-privilege violations

Skill: Mitigate Risk · **Maps to:** 5.2.A, 5.2.B · **Language:** Python 3 (pytest)

Problem

Build the audit function a security reviewer would run over a company's role assignments. Given a map of roles to the permissions they grant, a map of users to their assigned roles, and a map of users to the minimum permission set their job function actually requires, find every user who holds at least one permission (via their assigned roles) that is NOT in their job function's required set.

Function contract

Implement `solve(roles: dict, user_roles: dict, job_requirements: dict) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `roles`: `dict[str, list[str]]`, role name -> list of permissions it grants.
- `user_roles`: `dict[str, list[str]]`, user name -> list of role names assigned to them.
- `job_requirements`: `dict[str, list[str]]`, user name -> list of permissions their job function actually requires.

Output

A dict mapping each violating user to the **sorted** list of their excess (unauthorized) permissions. Users with no excess permissions are omitted entirely from the result.

Examples

```
roles = {"support": ["read_ticket", "write_ticket_reply"], "billing": ["read_invoice", "issue_refund"]}
user_roles = {"dana": ["support", "billing"]}
job_requirements = {"dana": ["read_ticket", "write_ticket_reply"]}

solve(roles, user_roles, job_requirements)
-> {"dana": ["issue_refund", "read_invoice"]}
```

```
roles = {"support": ["read_ticket"]}
user_roles = {"sam": ["support"]}
job_requirements = {"sam": ["read_ticket"]}

solve(roles, user_roles, job_requirements)
-> {}
```

Constraints

- Deterministic, offline, standard library only.
- A user with roles granting no permissions beyond their job requirements is omitted from the output entirely (not included with an empty list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import Dict, List


def solve(
    roles: Dict[str, List[str]],
```

```
    user_roles: Dict[str, List[str]],
    job_requirements: Dict[str, List[str]],
) -> Dict[str, List[str]]:
    result: Dict[str, List[str]] = {}
    for user, assigned_roles in user_roles.items():
        granted = set()
        for role in assigned_roles:
            granted.update(roles.get(role, []))
        required = set(job_requirements.get(user, []))
        excess = granted - required
        if excess:
            result[user] = sorted(excess)
    return result
```

5.2.c · Credential theft prevention and lateral movement

3 pts · auto-graded on the site

Coding problem, 5.2.c: Detect lateral movement in an authentication log

Skill: Mitigate Risk, Detect Attacks · **Maps to:** 5.2.B, 5.2.C · **Language:** Python 3 (pytest)

Problem

Build the detector a security team's SIEM rule would run over authentication events. Given a list of authentication events (account, destination host, timestamp in minutes), flag any account that authenticated to **at least `threshold` distinct hosts** within any rolling **`window_minutes`** window, a classic lateral-movement signal.

Function contract

Implement `solve(events: list, threshold: int, window_minutes: int) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `events`: `list[dict]`, each `{"account": str, "host": str, "ts": int}` (`ts` = minutes since start of log, non-negative integer, not necessarily sorted).
- `threshold`: `int`, minimum number of **distinct** hosts within the window to flag the account.
- `window_minutes`: `int`, size of the rolling time window, in minutes (inclusive on both ends).

Output

A **sorted, de-duplicated** `list[str]` of account names that hit the threshold at any point.

Examples

```
events = [
  {"account":"mkt_dana","host":"h1","ts":0},
  {"account":"mkt_dana","host":"h2","ts":2},
  {"account":"mkt_dana","host":"h3","ts":4},
  {"account":"svc_backup","host":"h9","ts":0},
]
solve(events, threshold=3, window_minutes=5) -> ["mkt_dana"]
```

```
events = [
  {"account":"alice","host":"h1","ts":0},
  {"account":"alice","host":"h2","ts":100},
]
solve(events, threshold=2, window_minutes=5) -> []   # too far apart for the window
```

Constraints

- Deterministic, offline, standard library only.
- A window `[t, t + window_minutes]` counts an event if its `ts` falls within that inclusive range; consider every event's `ts` as a possible window start.
- Repeated authentications to the SAME host do not add distinct-host count.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from collections import defaultdict
from typing import Dict, List


def solve(events: List[Dict[str, object]], threshold: int, window_minutes: int) -> List[str]:
    by_account: Dict[str, List[Dict[str, object]]] = defaultdict(list)
    for ev in events:
        by_account[ev["account"]].append(ev)

    flagged = set()
    for account, evs in by_account.items():
        starts = sorted(set(e["ts"] for e in evs))
        for start in starts:
            window_hosts = set(
                e["host"] for e in evs if start <= e["ts"] <= start + window_minutes
            )
            if len(window_hosts) >= threshold:
                flagged.add(account)
                break

    return sorted(flagged)
```